

Relazione di Analisi e Trattamento dei segnali digitali

Bordis Paolo Alberto

July 3, 2025

Contents

1 Introduzione **3**

2 Preparazione e filtraggio del segnale **4**
2.1 Lettura dei dati 4
2.2 Normalizzazione del segnale 5
2.3 Progettazione del primo filtro 5
2.4 Progettazione filtro mediano e butter-worth 7

3 Rilevamento frequenza cardiaca **10**
3.1 Filtraggio R-peaks e segnale energia 10
3.2 Ulteriore filtraggio usando il filtro integratore 11
3.3 Algoritmo per la frequenza cardiaca 12

4 Conclusioni **15**

1 Introduzione

L'elettrocardiogramma (ECG) è uno strumento fondamentale per il monitoraggio e la diagnosi delle condizioni cardiache, poiché consente di registrare in modo non invasivo l'attività elettrica del cuore.

In questa relazione verrà analizzato un segnale ECG fornito, con l'obiettivo di studiarlo e ripulirlo dalle componenti indesiderate che ne compromettono la qualità, utilizzando tecniche di filtraggio digitale.

Inoltre, verrà implementato un algoritmo di rilevamento dei picchi per derivare la frequenza cardiaca dal segnale. Questo algoritmo sarà infine utilizzato per confrontare i risultati ottenuti su due segnali: uno filtrato, privo delle componenti disturbanti, e uno non filtrato, contenente tali disturbi.

Ma perché è necessario filtrare il segnale? Durante l'acquisizione del tracciato ECG, oltre all'attività elettrica del cuore, vengono campionate anche altre componenti indesiderate, come il rumore generato dal macchinario, le interferenze elettriche esterne (come il rumore di rete), e le frequenze legate alla respirazione o ai movimenti del paziente. Questi disturbi, se non corretti, possono ostacolare l'analisi clinica e ridurre l'affidabilità della diagnosi.

Va ricordato, inoltre, che l'ECG non è costituito da un singolo segnale, bensì da cinque onde distinte, di cui tre sono comunemente classificate come onde positive, mentre le restanti due sono considerate onde negative.

Queste onde rappresentano le diverse fasi del ciclo cardiaco e sono identificate con le lettere **PQRST**: le prime tre (P, Q, R) sono di tipo positivo, mentre le ultime due (S, T) sono considerate negative. Ognuna di esse fornisce informazioni preziose sul funzionamento del muscolo cardiaco.

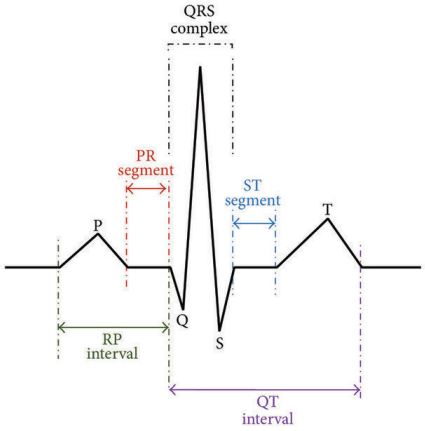


Figure 1: Schema di un elettrocardiogramma (ECG) con la rappresentazione delle onde PQRST.

2 Preparazione e filtraggio del segnale

2.1 Lettura dei dati

```
% Analisi ECG
close all
clear all

% Lettura del dataset
filename = 'rec_1.dat';
fid = fopen(filename, 'r');
ecg = fread(fid, Inf, 'int16');
fclose(fid);

% Frequenza di campionamento e frequenza di Nyquist
fs = 500;
fn = fs/2;
```

Listing 1: Caricamento dei dati.

Inanzitutto, per iniziare un'analisi di questo tipo, è necessario disporre di un segnale già digitalizzato. Il segnale analizzato è stato scaricato da uno dei repository più accreditati per quanto riguarda i segnali biomedicali digitali, ovvero PhysioNet ECG-ID Database.

Una volta scaricato il file `.dat`, è possibile leggerlo utilizzando le istruzioni MATLAB come `fopen` e `fread`.

Un altro parametro fondamentale da considerare è la frequenza di campionamento (`fs`), che viene fornita direttamente da chi ha pubblicato il dataset.

Una volta trovata la frequenza di campionamento, noi avremo che la frequenza di nysquit sarà sempre la metà della frequenza di campionamento, questo per diretta conseguenza del teorema del campionamento.

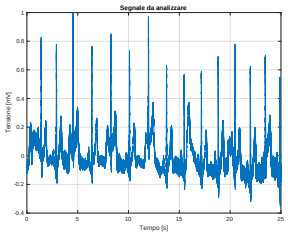


Figure 2: Plotting del Segnale Raw prelevato.

Questo è il risultato del segnale caricato, come possiamo notare abbiamo parecchio rumore è il segnale si presenta non tanto asciutto ma con componenti che lo ingrossano parecchio rispetto alla forma che noi generalmente ci aspettiamo di un ECG epurato.

2.2 Normalizzazione del segnale

Innanzitutto, si effettua una selezione sul vettore ECG importato. È necessario normalizzare il segnale in funzione del tempo, calcolando il numero totale di campioni e dividendolo per la frequenza di campionamento. In questo modo si ottiene un vettore temporale `t` che rappresenta i secondi corrispondenti a ciascun campione.

Una volta ottenuta la corrispondenza tempo-campione, è possibile estrarre i primi 5 secondi del segnale, selezionando i valori di `ecg` compresi nell'intervallo temporale desiderato.

```
% Preparazione del segnale e normalizzazione
n_samples = length(ecg) - 1;
t = (0:n_samples)/fs;

% Prelevo i primi 5 secondi
t = t(t <= 5);
ecg_1 = ecg(1:length(t));

% Normalizzazione e rimozione del valore medio
ecg_1 = (ecg_1 - min(ecg_1)) / (max(ecg_1) - min(ecg_1));
ecg_1 = ecg_1 - mean(ecg_1);
```

Listing 2: Normalizzazione con rimozione del valor medio.

Successivamente effettuo la normalizzione, però lo faccio rispetto a tutto il dataset e non rispetto ai campioni che sto considerando.

2.3 Progettazione del primo filtro

```
% Progettazione del filtro ellittico per eliminare la
    componente a 50 Hz
Wp = [46 54]/fn; % Banda passante vicina a 50 Hz
Ws = [48 52]/fn; % Banda di stop centrata su 50 Hz
Rp = 1;          % Attenuazione massima nella banda
    passante (dB)
Rs = 50;         % Attenuazione minima nella banda di stop
    (dB)
```

Listing 3: Progettazione del primo filtro.

Il primo filtro progettato ha lo scopo di rimuovere la frequenza di disturbo a 50 Hz, comunemente associata al rumore della rete elettrica (power line interference o power supply noise).

Inizialmente, si era pensato di utilizzare un semplice filtro **stop-band**, in grado di eliminare completamente la componente a 50 Hz. Tuttavia, questo ap-

proccio si è rivelato poco efficace, poiché tendeva ad attenuare anche le frequenze adiacenti, compromettendo parti utili del segnale ECG.

```
% Calcolo dell'ordine ottimale del filtro
[n, Wp] = ellipord(Wp, Ws, Rp, Rs);
[z, p, k] = ellip(n, Rp, Rs, Wp, 'stop');
[b, a] = zp2tf(z, p, k);
```

Listing 4: Realizzazione del filtro con i parametri precedentemente definiti.

Una volta impostati i parametri, si utilizza la funzione `ellipord` per determinare l'ordine minimo del filtro ellittico che soddisfi le specifiche richieste. Successivamente, con la funzione `ellip` vengono calcolati zeri, poli e guadagno del filtro, che vengono poi convertiti nella rappresentazione polinomiale della funzione di trasferimento tramite `zp2tf`.

Il passo successivo consiste nell'analizzare il comportamento del filtro, utilizzando i comandi `freqz` e `zplane`, per visualizzarne la risposta in frequenza e il diagramma dei poli e degli zeri.

Ci si aspetta di osservare una banda di attenuazione stretta attorno a 0,2 rad/s (determinata da $\frac{50\,\text{Hz}}{f_n}$), mentre il resto dello spettro dovrebbe risultare invariato.

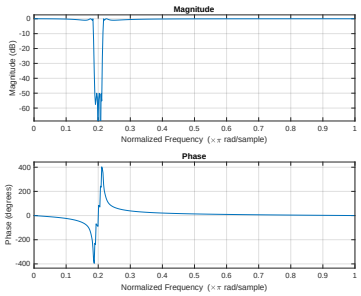


Figure 3: Risposta in frequenza del filtro ellittico centrato a 50 Hz.

Infine, analizziamo il diagramma di poli e zeri, il quale fornisce una rappresentazione visiva della struttura del filtro. In particolare, ci aspettiamo di osservare la presenza di zeri (indicati con delle `x`) in prossimità della frequenza normalizzata pari a 0,2 rad/s.

Questi zeri sono posizionati strategicamente per attenuare la banda indesiderata attorno ai 50 Hz, senza compromettere significativamente le altre componenti del segnale. Il comportamento del modulo della risposta in frequenza è dunque coerente con la distribuzione degli zeri e dei poli nel piano z.

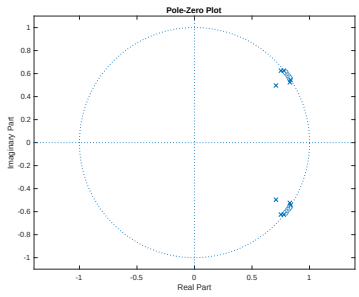


Figure 4: Diagramma di poli e zeri del filtro ellittico progettato.

2.4 Progettazione filtro mediano e butter-worth

Infine procediamo con la creazione degli ultimi due filtri, uno quello mediano e l'altro è un filtro passa alto di secondo ordine per il filtraggio di 0.5 Hz.

```
% Filtro mediano di ordine 10
y2 = medfilt1(y, 10);

% Filtro Butterworth passa-alto di secondo ordine per
    frequenza di taglio 0.5 Hz
[b1, a1] = butter(2, (0.5/fn), 'high');

y3 = filter(b1, a1, y2);
```

Listing 5: Filtraggio con filtro mediano e realizzazione del filtro butter-worth.

Andiamo a vedere diagramma di poli e zeri e della risposta all'impulso del filtro butter.

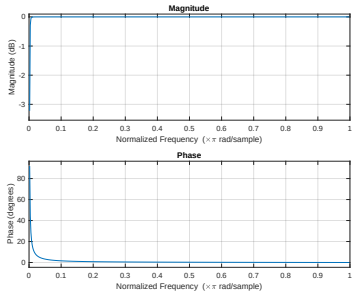


Figure 5: Risposta all'impulso del filtro butter.

Come possiamo notare la risposta all'impulso penalizza le piccole frequenze, ovvero le frequenze di 0.5 rad/s e quelle vicine, mentre mantiene un guadagno alto su tutte quelle maggiori di quest'ultima.

Ora vediamo il risultato finale andando a fare il confronto con il segnale grezzo trovato precedentemente.

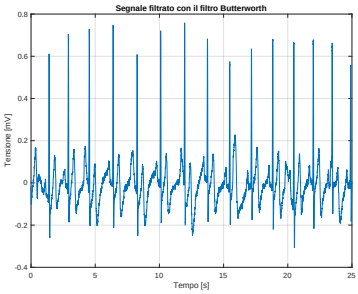


Figure 6: Segnale filtrato.

Ora, invece, osserviamo che il segnale presentato ha una forma più pulita e definita rispetto a quello grezzo prelevato dal file; la sagoma del segnale rispecchia ora quella di un vero tracciato ECG.

3 Rilevamento frequenza cardiaca

La seconda parte di questa relazione riguarda l'implementazione di un algoritmo per la rilevazione della frequenza cardiaca. In particolare, l'obiettivo è confrontare la frequenza cardiaca ottenuta da un segnale grezzo (raw), ovvero non filtrato e contenente tutte le componenti di disturbo, con quella ottenuta da un segnale filtrato.

3.1 Filtraggio R-peaks e segnale energia

Come da titolo al fine di poter computare la rilevazione della frequenza cardiaca sono necessari due filtraggi.[1] Questi due filtri sono di tipo butter-worth di ordine quattro.

```
1
2         [b2, a2] = butter(4, 15/250, 'high');
3         freqz(b2, a2);
4
5         [b3, a3] = butter(4, 20/250, 'low');
6         freqz(b3, a3);
```

Listing 6: R - peaks filtering.

Viene chiamato **R-peaks filtering** proprio perché è un tipo di filtraggio che serve per la rilevazione e il calcolo della frequeza cardiaca.

Successivamente deriviamo il segnale energia, dove ricordiamo che matematicamente il segnale energia è dato dalla formula

$$E(t) = u(t)^2$$

e dove ricordiamo che la formula su MATLAB per ottenerlo è data da `energy = abs(segnale).^2`.

Vediamo ora graficamente a confronto il segnale energia tra quello grezzo e quello ripulito.

[1] Sono necessari due filtri, e non un unico filtro di tipo stop-band, perché, come riportato nell'articolo di riferimento, utilizzare due filtri impostati in questo modo è computazionalmente più vantaggioso rispetto all'impiego di un solo filtro che copra l'intera banda.

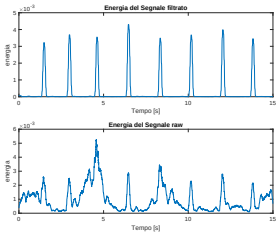


Figure 7: Plotting dei due segnali energia.

Ovviamente si nota una marcata differenza tra i plot dei segnali: l'effetto dei primi filtraggi produce un segnale di energia particolarmente pulito, con componenti a bassa frequenza ridotte, ad eccezione delle zone in corrispondenza dei picchi di energia, che rappresentano l'onda R.

3.2 Ulteriore filtraggio usando il filtro integratore

Ora invece ci dedichiamo ad una parte utile per lo svilippo dell'algoritmo per la determinazione della frequenza cardiaca. Per prima cosa, dobbiamo sviluppare un filtro integratore, che ci servirà per ripulire ulteriormente il segnale e per amplificare gli accumuli di energia, garantendo un guadagno maggiore soprattutto per le componenti a bassa frequenza.

```
1         window_len = round(0.150 * fs);
2         envelope = conv(energy, ones(1, window_len) / window_len, '
              same');
3         envelope_1 = conv(energy_1, ones(1, window_len) /
              window_len, 'same');
```

Listing 7: Sviluppo del segnale integratore e applicazione di esso.

Una volta ottenuto il segnale prodotto dalla convoluzione del filtro integratore e il segnale R-peaks filtrato, non ci resta che presentare l'algoritmo per la rilevazione della frequenza cardiaca.

3.3 Algoritmo per la frequenza cardiaca

```
function [qrs_peaks, locs, r_times, heart_rate, hr_time] =
    heart_rate_detection(signal, fs)

age = 25;
max_HR_bpm = 220 - age;
min_RR = 60 / max_HR_bpm;
min_samples = round(min_RR * fs);

[qrs_peaks, locs] = findpeaks(signal, ...
'MinPeakDistance', min_samples);

t = (0:length(signal)-1) / fs;
r_times = t(locs);

rr_intervals = diff(r_times);
heart_rate = 60 ./ rr_intervals;
hr_time = r_times(2:end);

end
```

Listing 8: Algoritmo per la detection della frequenza cardiaca.

L'algoritmo prende in input il segnale elaborato tramite i filtraggi descritti in precedenza, la sua frequenza di campionamento e il segnale originale. Restituisce le ampiezze dei picchi R individuati, le loro posizioni (in campioni), i relativi istanti temporali (in secondi), la sequenza dei valori di frequenza cardiaca (in BPM) e i corrispondenti istanti temporali associati a tali valori.

La principale chiamata di funzione utilizzata in questo algoritmo è `findpeaks`, che restituisce i principali picchi del segnale da analizzare. In particolare, è stata configurata per rilevare picchi distanti almeno quanto il minimo intervallo fisiologico tra due battiti, dove l'intervallo minimo fisiologico è definito come

$$\texttt{min_RR} = \frac{60}{\texttt{max_HR_bpm}}.$$

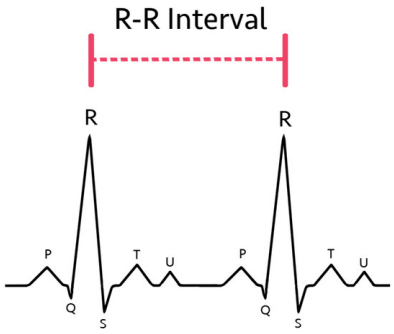


Figure 8: Intervallo RR.

Esattamente come in figura, vogliamo prima determinare i picchi tra le due onde R o tra set di onde; successivamente calcoliamo le differenze di tempi con il comando

$$\texttt{rr_intervals} = \texttt{diff(r_times)}.$$

Infine, possiamo determinare la frequenza cardiaca istantanea, espressa in battiti per minuto (BPM), usando la seguente formula:

$$\texttt{heart_rate} = 60./\texttt{rr_intervals}.$$

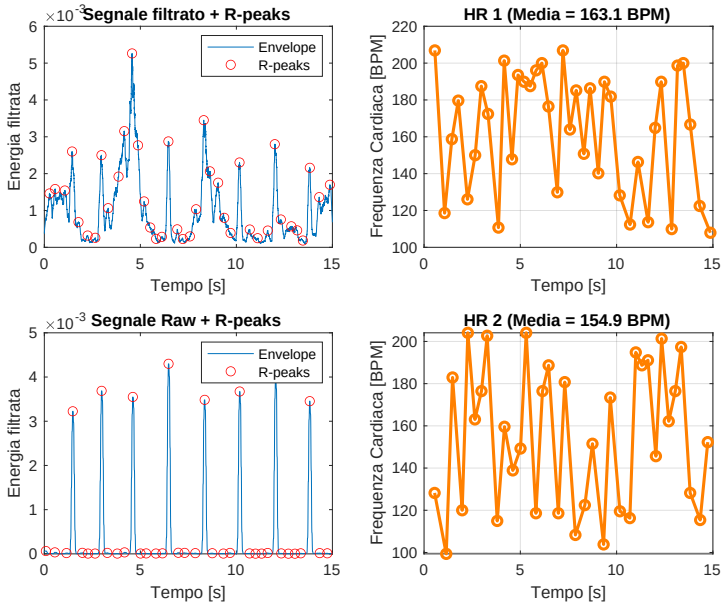


Figure 9: Plotting dei due segnali energia.

4 Conclusioni

Notiamo come, mettendo a confronto i due casi, si abbiano due diverse applicazioni della funzione **findpeaks**: da una parte essa rileva i picchi su un segnale completamente pulito, mentre dall'altra individua un numero maggiore di picchi, poiché l'ampiezza di alcune componenti residue — che non rappresentano effettivamente l'onda **R** — può superare la soglia di rilevamento.

È comunque interessante osservare che il valore medio della frequenza cardiaca non si discosta significativamente da quello calcolato sul segnale filtrato.

Infine, è bene sottolineare che il segnale analizzato (quello raw), pur mostrando una media della frequenza cardiaca con una differenza di circa una decina di battiti, non rappresenta un'informazione attendibile.

Questo perché, per determinare correttamente la frequenza cardiaca, è necessario calcolare le differenze temporali tra le onde R. Tuttavia, i picchi individuati nel grafico del segnale raw non coincidono sempre con le reali onde R, rendendo quindi poco significativo il risultato restituito dall'algoritmo.